

Basics of Functional Dependencies and Normalization for Relational Databases.

Four informal guidelines that may be used as measures to determine the quality of relation schema design:

- ### 1.1 Imparting Clear Semantics to Attributes in Relations.

- | EMPLOYEE | | | | | F.K. |
|----------|------------|-------|---------|---------|------|
| Ename | <u>Ssn</u> | Bdate | Address | Dnumber | |
| | | | | | P.K. |

DEPARTMENT			F.K.
Dname	<u>Dnumber</u>	Dmgr_ssn	
			P.K.

- Design a relation schema so that it is easy to explain its meaning.
- Do not combine attributes from multiple entity types and relationship types into a single relation.
- If a relation schema corresponds to one entity type or one relationship type, it is straightforward to interpret and to explain its meaning. Otherwise, if the relation

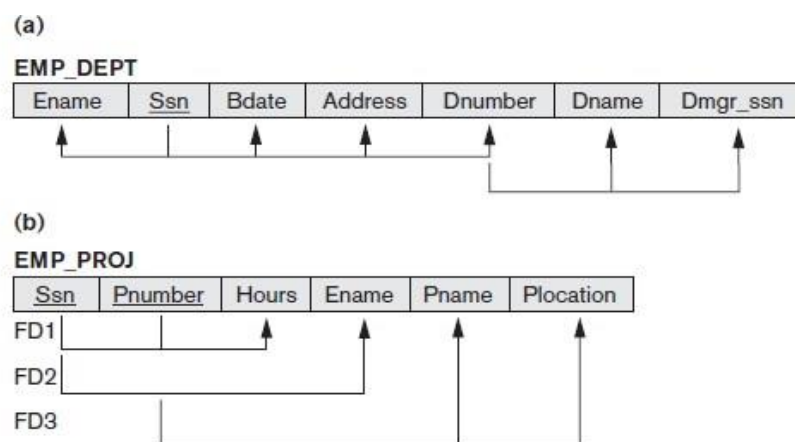
corresponds to a mixture of multiple entities and relationships, semantic ambiguities will result and the relation cannot be easily explained.

Examples of Violating Guideline 1.

- The following relation schema EMP_DEPT and EMP_PROJ have clear semantics but they violate Guideline 1 by mixing attributes from distinct real-world entities: EMP_DEPT mixes attributes of employees and departments, and EMP_PROJ mixes attributes of employees and projects and the WORKS_ON relationship. Hence, they fare poorly against the above measure of design quality.

Figure 15.3

Two relation schemas suffering from update anomalies. (a) EMP_DEPT and (b) EMP_PROJ.



1.2 Redundant Information in Tuples and Update Anomalies.

- One goal of schema design is to minimize the storage space used by the base relations. Grouping attributes into relation schemas has a significant effect on storage space.
- For example, The space used by the two base relations **EMPLOYEE** and **DEPARTMENT** is less compared to **EMP_DEPT**.

EMPLOYEE

Ename	<u>Ssn</u>	Bdate	Address	Dnumber
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4
Wallace, Jennifer S.	987654321	1941-06-20	291Berry, Bellaire, TX	4
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn
Research	5	333445555
Administration	4	987654321

Database Management System

1. In **EMP_DEPT**, the attribute values pertaining to a particular department (Dnumber, Dname, Dmgr_ssn) are repeated for every employee who works for that department. In contrast, each department's information appears only once in the **DEPARTMENT** relation.

Redundancy

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555

2. EMP_DEPT base relation is the result of applying the NATURAL JOIN operation to EMPLOYEE and DEPARTMENT. Storing natural joins of base relations leads to an additional problem referred to as **update anomalies**.

Update anomalies can be classified into **insertion anomalies**, **deletion anomalies**, and **modification anomalies**.

Insertion Anomalies.

Insertion anomalies can be differentiated into two types, based on the EMP_DEPT relation:

1. To insert a new employee tuple into EMP_DEPT, we must include either the attribute values for the department that the employee works for, or NULLs (if the employee does not work for a department as yet).
2. It is difficult to insert a new department that has no employees as yet in the EMP_DEPT relation. The only way to do this is to place NULL values in the attributes for employee. This violates the entity integrity for EMP_DEPT because Ssn is its primary key.

Deletion Anomalies.

The problem of deletion anomalies is related to the second insertion anomaly situation.

1. If we delete from EMP_DEPT an employee tuple that happens to represent the last employee working for a particular department, the information concerning that department is lost from the database.
2. This problem does not occur in DEPARTMENT relation since tuples are stored separately.

Modification Anomalies.

1. In EMP_DEPT, if we change the value of one of the attributes of a particular department say, the manager of department 5 we must update the tuples of all employees who work in that department; otherwise, the database will become inconsistent.
2. If we fail to update some tuples, the same department will be shown to have two different values for manager in different employee tuples, which would be wrong.

Guideline 2

- Design the base relation schemas so that no insertion, deletion, or modification anomalies are present in the relations.
- If any anomalies are present, note them clearly and make sure that the programs that update the database will operate correctly.

1.3 NULL Values in Tuples

- If many of the attributes do not apply to all tuples in the relation, we end up with many NULLs in those tuples. This can waste space at the storage level and may also lead to problems with understanding the meaning of the attributes.
- SELECT and JOIN operations involve comparisons; if NULL values are present, the results may become unpredictable.
- NULLs can have multiple interpretations, such as the following:
 1. The attribute does not apply to this tuple. For example, Visa_status may not apply to U.S. students.
 2. The attribute value for this tuple is unknown. For example, the Date_of_birth may be unknown for an employee.
 3. The value is known but absent; For example, the Home_Phone_Number for an employee may exist, but may not be available and recorded yet.

Guideline 3

- Avoid placing attributes in a base relation whose values may frequently be NULL. If NULLs are unavoidable, make sure that they apply in exceptional cases only and do not apply to a majority of tuples in the relation.

1.4 Generation of Spurious Tuples

- Consider the two relation schemas EMP_LOCS and EMP_PROJ1. Suppose if we perform NATURAL JOIN operation on EMP_PROJ1 and EMP_LOCS, the result produces many more tuples than the original set of tuples .
- These additional tuples are called **spurious tuples** because they represent spurious information that is not valid. The spurious tuples are marked by asterisks (*) in Figure 15.6.

Database Management System

- Decomposing EMP_PROJ into EMP_LOCS and EMP_PROJ1 is undesirable because when we JOIN them back using NATURAL JOIN, we do not get the correct original information. This is because in this case Plocation is the attribute that relates EMP_LOCS and EMP_PROJ1, and Plocation is neither a primary key nor a foreign key in either EMP_LOCS or EMP_PROJ1.

EMP_LOCS

Ename	Plocation
Smith, John B.	Bellaire
Smith, John B.	Sugarland
Narayan, Ramesh K.	Houston
English, Joyce A.	Bellaire

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation
123456789	1	32.5	ProductX	Bellaire
123456789	2	7.5	ProductY	Sugarland
666884444	3	40.0	ProductZ	Houston

	Ssn	Pnumber	Hours	Pname	Plocation	Ename
	123456789	1	32.5	ProductX	Bellaire	Smith, John B.
*	123456789	1	32.5	ProductX	Bellaire	English, Joyce A.
	123456789	2	7.5	ProductY	Sugarland	Smith, John B.
*	123456789	2	7.5	ProductY	Sugarland	English, Joyce A.
*	123456789	2	7.5	ProductY	Sugarland	Wong, Franklin T.
	666884444	3	40.0	ProductZ	Houston	Narayan, Ramesh K.

Guideline 4

- Design relation schemas so that they can be joined with equality conditions on attributes that are appropriately related (primary key, foreign key) pairs in a way that guarantees that no spurious tuples are generated.
- Avoid relations that contain matching attributes that are not (foreign key, primary key) combinations because joining on such attributes may produce spurious tuples.

2 Functional Dependencies

- Definition :** A **functional dependency**, denoted by $X \rightarrow Y$, between two sets of attributes X and Y that are subsets of R specifies a constraint on the tuples in a relation state r of R. The constraint is that, for any two tuples t1 and t2 in r that have $t1[X] = t2[X]$, they must also have $t1[Y] = t2[Y]$.
- A **functional dependency**, denoted by $X \rightarrow Y$ means that the values of the Y are determined by the values of X .
- A functional dependency is a property of the **semantics** or **meaning of the attributes**. The database designers will use their understanding of the semantics of the attributes of R to specify the functional dependencies in a relation.
- Consider the relation schema EMP_PROJ from the semantics of the attributes and the relation, the following functional dependencies should hold:

- a. $Ssn \rightarrow Ename$
- b. $Pnumber \rightarrow \{Pname, Plocation\}$
- c. $\{Ssn, Pnumber\} \rightarrow Hours$
- These functional dependencies specifies that
 - (a) the value of an employee's Social Security number (Ssn) uniquely determines the employee name (Ename),
 - (b) the value of a project's number (Pnumber) uniquely determines the project name (Pname) and location (Plocation), and
 - (c) Combination of Ssn and Pnumber values uniquely determines the number of hours the employee currently works on the project per week (Hours).
- Types of functional dependency :
 1. A functional dependency $X \rightarrow Y$ is a **full functional dependency** if removal of any attribute A from X means that the dependency does not hold any more;
Ex: **$\{Ssn, Pnumber\} \rightarrow Hours$** is a full dependency (neither $Ssn \rightarrow Hours$ nor $Pnumber \rightarrow Hours$ holds).
 2. A functional dependency $X \rightarrow Y$ is a **partial functional dependency** if removal of any attribute A from X and the dependency still holds;
Ex: **$\{Ssn, Pnumber\} \rightarrow Ename$** is partial because $Ssn \rightarrow Ename$ holds.
 3. A functional dependency $X \rightarrow Y$ in a relation schema R is a **transitive dependency** if there exists a set of attributes Z in R such that $X \rightarrow Z$ and $Z \rightarrow Y$ hold.
Ex: The dependency $Ssn \rightarrow Dmgr_ssn$ is transitive in EMP_DEPT, because of the dependencies $Ssn \rightarrow Dnumber$ and $Dnumber \rightarrow Dmgr_ssn$.
 4. **Trivial Functional Dependency.** If a **functional dependency (FD)** $X \rightarrow Y$ holds, where Y is a subset of X , then it is called a **trivial FD**.
Non-trivial – If an FD $X \rightarrow Y$ holds, where Y is not a subset of X , then it is called a **non-trivial FD**.

3 Normal Forms Based on Primary Keys

3.1 Normalization of Relations

- The normalization process, as first proposed by Codd (1972). Codd proposed three normal forms, which he called first, second, third normal form and Boyce-Codd normal form (BCNF). All these normal forms are based on functional dependencies among the attributes of a relation. Later, a fourth normal form (4NF) and a fifth normal form (5NF) were proposed, based on the concepts of multivalued dependencies and join dependencies, respectively;
- **Normalization of data** can be considered a process of analyzing the given relation schemas based on their FDs and primary keys to achieve the desirable properties of (1) minimizing redundancy and (2) minimizing the insertion, deletion, and update anomalies.
- It can be considered as a “filtering” or “purification” process to make the design have successively better quality.
- **Definition.** The **normal form** of a relation refers to the highest normal form condition that it meets, and hence indicates the degree to which it has been normalized.

3.2 Definitions of Keys and Attributes Participating in Keys.

- **Definition.** A **superkey** of a relation schema R is a set of attributes $S \subseteq R$ with the property that no two tuples t_1 and t_2 in any legal relation state r of R will have $t_1[S] = t_2[S]$.
- A **key** K is a superkey with the additional property that removal of any attribute from K will cause K not to be a superkey any more.
- The difference between a key and a superkey is that a key has to be *minimal*; that is, if we have a key $K = \{A_1, A_2, \dots, A_k\}$ of R , then $K - \{A_i\}$ is not a key of R .
Ex: $\{Ssn\}$ is a key for EMPLOYEE, whereas $\{Ssn\}$, $\{Ssn, Ename\}$, $\{Ssn, Ename, Bdate\}$, and any set of attributes that includes Ssn are all superkeys.
- If a relation schema has more than one key, each is called a **candidate key**. One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called secondary keys. $\{Ssn\}$ is the only candidate key for EMPLOYEE, so it is also the primary key.
- **Definition.** An attribute of relation schema R is called a **prime attribute** of R if it is a member of some candidate key of R . An attribute is called **nonprime** if it is not a prime attribute—that is, if it is not a member of any candidate key.
Ex: Ssn and $Pnumber$ are prime attributes of WORKS_ON, whereas other attributes of WORKS_ON are nonprime.

3.3 First Normal Form

- **First normal form (1NF)** states that the domain of an attribute must include only *atomic* (simple, indivisible) *values* and that the value of any attribute in a tuple must be a *single value* from the domain of that attribute.
- Consider the following DEPARTMENT relation, It is not in 1NF. Because the domain of Dlocations contains sets of values and hence is nonatomic.

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

- There are three main techniques to achieve first normal form for such a relation:
 1. Remove the attribute Dlocations that violates 1NF and place it in a separate relation DEPT_LOCATIONS along with the primary key Dnumber of DEPARTMENT. The primary key of this relation is the combination $\{Dnumber, Dlocation\}$.

Database Management System

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

2. Expand the key so that there will be a separate tuple in the original DEPARTMENT relation for each location of a DEPARTMENT. In this case, the primary key becomes the combination {Dnumber, Dlocation}. This solution has the disadvantage of introducing *redundancy* in the relation.

DEPARTMENT

<u>Dname</u>	<u>Dnumber</u>	<u>Dmgr_ssn</u>	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

3. If a maximum number of values is known for the attribute for example, if it is known that at most three locations can exist for a department—replace the Dlocations attribute by three atomic attributes: Dlocation1, Dlocation2, and Dlocation3. This solution has the disadvantage of introducing NULL values if most departments have fewer than three locations.
- First normal form also disallows multivalued attributes that are themselves composite. These are called **nested relations** because each tuple can have a relation *within it*.
 - Figure 15.10 the EMP_PROJ relation represents an employee entity, and a relation PROJS(Pnumber, Hours) *within each tuple*.
 - The schema of this EMP_PROJ relation can be represented as follows: EMP_PROJ(Ssn, Ename, {PROJS(Pnumber, Hours)}). The set braces { } identify the attribute PROJS as multivalued, and we list the component attributes that form PROJS between parentheses ().
 - To normalize this into 1NF, we remove the nested relation attributes into a new relation and *propagate the primary key* into it; Decomposition and primary key propagation yield the schemas EMP_PROJ1 and EMP_PROJ2, as shown in Figure 15.10(c).

(a)

EMP_PROJ		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ			
Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0

(c)

EMP_PROJ1	
Ssn	Ename

EMP_PROJ2		
Ssn	Pnumber	Hours

Figure 15.10 Normalizing nested relations into 1NF.

- (a) Schema of the EMP_PROJ relation with a *nested relation* attribute PROJS.
- (b) Sample extension of the EMP_PROJ relation showing nested relations within each tuple.
- c) Decomposing EMP_PROJ into EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

3.4 Second normal form (2NF)

- **Definition.** A relation schema R is in 2NF if every nonprime attribute A in R is **fully functionally dependent** on the primary key of R .
- Second normal form (2NF) is based on the concept of full functional dependency. A functional dependency $X \rightarrow Y$ is a **full functional dependency** if removal of any attribute A from X means that the dependency does not hold any more.
Example1: $\{Ssn, Pnumber\} \rightarrow Hours$ is a full dependency (neither $Ssn \rightarrow Hours$ nor $Pnumber \rightarrow Hours$ holds).
- A functional dependency $X \rightarrow Y$ is a **partial functional dependency** if removal of any attribute A from X and the dependency still holds;
Example2: The dependency $\{Ssn, Pnumber\} \rightarrow Ename$ is partial because $Ssn \rightarrow Ename$ holds.
- The EMP_PROJ relation is in 1NF but is not in 2NF. The nonprime attribute Ename violates 2NF because of FD2, Pname and Plocation violates 2NF because of FD3.
- The functional dependencies FD2 and FD3 make Ename, Pname, and Plocation partially dependent on the primary key $\{Ssn, Pnumber\}$ of EMP_PROJ, thus violating the 2NF test.

- If a relation schema is not in 2NF, it can be *second normalized by decomposing* EMP_PROJ into the three relation schemas EP1, EP2, and EP3 shown in Figure 15.11(a), each of which is in 2NF.

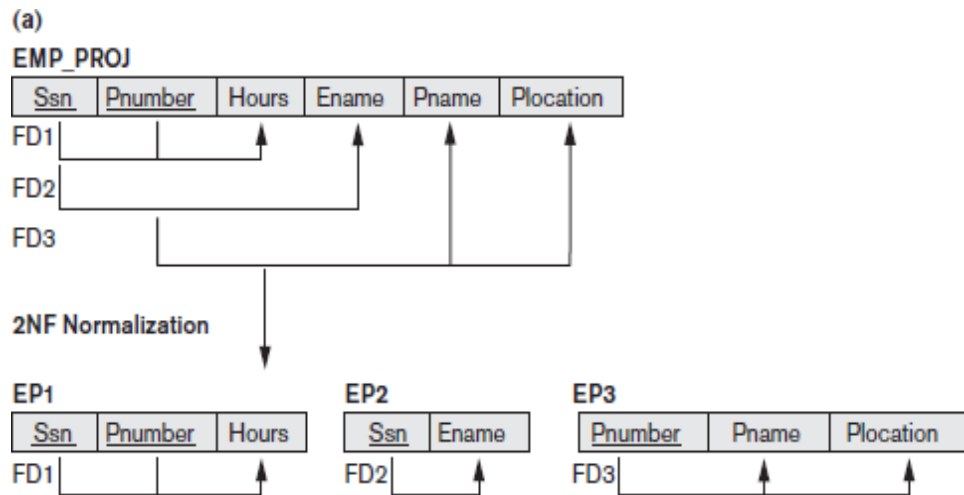


Figure 15.11- Normalizing into 2NF and 3NF.

(a) Normalizing EMP_PROJ into 2NF relations.

Example 2:

- Consider the relation schema LOTS shown in Figure 15.12(a). There are two candidate keys: Property_id# and {County_name, Lot#}; lot numbers are unique only within each county, but Property_id# numbers are unique across counties for the entire state.
- There are two candidate keys Property_id# and {County_name, Lot#}. We choose Property_id# as the primary key, so it is underlined in Figure 15.12(a).
- The LOTS relation schema violates the general definition of 2NF because Tax_rate is partially dependent on the candidate key {County_name, Lot#}, due to FD3.
- To normalize LOTS into 2NF, we decompose it into the two relations LOTS1 and LOTS2, shown in Figure 15.12(b). We construct LOTS1 by removing the attribute Tax_rate that violates 2NF from LOTS and placing it with County_name into another relation LOTS2. Both LOTS1 and LOTS2 are in 2NF.

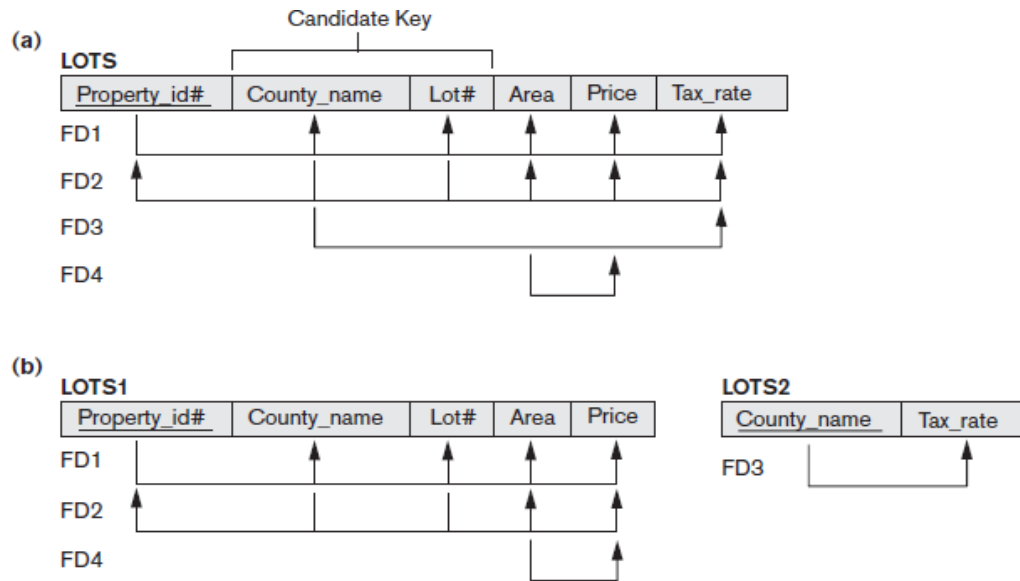


Figure 15.12

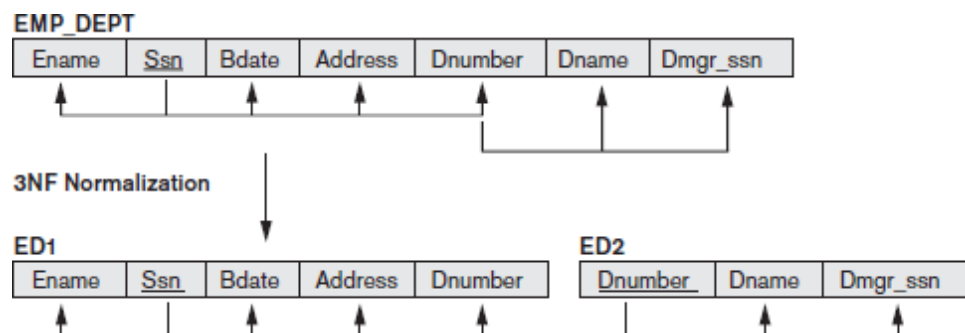
(a) The LOTS relation with its functional dependencies FD1 through FD4.

(b) Decomposing into the 2NF relations LOTS1 and LOTS2.

3.5 Third Normal Form (3NF)

- **Definition:** A relation schema R is in 3NF if it satisfies 2NF *and* no nonprime attribute of R is transitively dependent on the primary key.
- Third normal form (3NF) is based on the concept of transitive dependency. A functional dependency $X \rightarrow Y$ in a relation schema R is a **transitive dependency** if there exists a set of attributes Z in R such that $X \rightarrow Z$ and $Z \rightarrow Y$ hold.

Ex: The dependency $Ssn \rightarrow Dmgr_ssn$ is transitive in EMP_DEPT, because of the dependencies $Ssn \rightarrow Dnumber$ and $Dnumber \rightarrow Dmgr_ssn$.



Normalizing EMP_DEPT into 3NF relations.

- The relation schema EMP_DEPT is not in 3NF because of the transitive dependency of Dmgr_ssn and Dname on Ssn via Dnumber. We can normalize EMP_DEPT by decomposing it into the two 3NF relation schemas ED1 and ED2 shown in the above figure.

Example 2:

- FD4 in LOTS1 violates 3NF because Area is not a superkey and Price is not a prime attribute in LOTS1.
- To normalize LOTS1 into 3NF, we decompose it into the relation schemas LOTS1A and LOTS1B shown in Figure 15.12(c). We construct LOTS1A by removing the attribute Price that violates 3NF from LOTS1 and placing it with Area (the lefthand side of FD4 that causes the transitive dependency) into another relation LOTS1B. Both LOTS1A and LOTS1B are in 3NF.

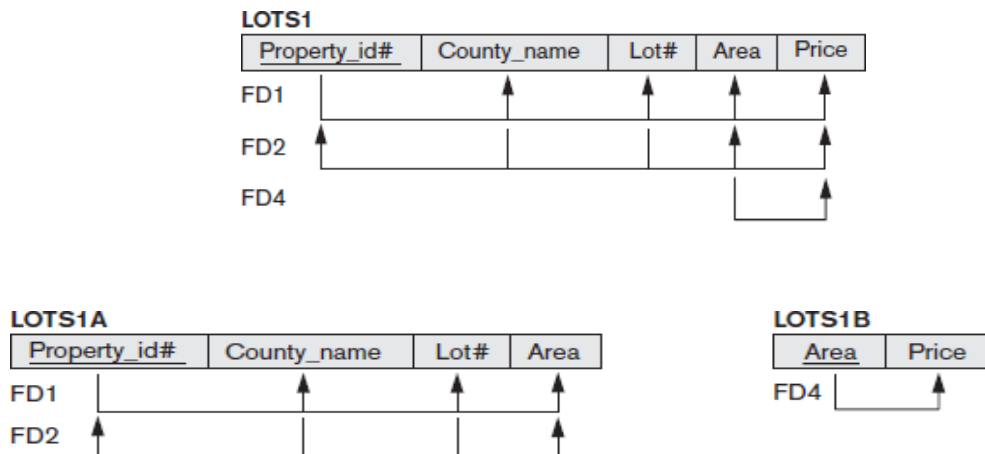


Figure 15.12(c). Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B.

4. Boyce-Codd Normal Form.

- Definition.** A relation schema R is in BCNF if whenever a *nontrivial* functional dependency $X \rightarrow A$ holds in R , then X is a superkey of R .
- The BCNF is based on the concept non trivial dependency. If an FD $X \rightarrow Y$ holds, where Y is not a subset of X , then it is called a non-trivial FD.
- FD5 violates BCNF in LOTS1A because AREA is not a superkey of LOTS1A. decompose LOTS1A into two BCNF relations LOTS1AX and LOTS1AY, shown in Figure 15.13(a).

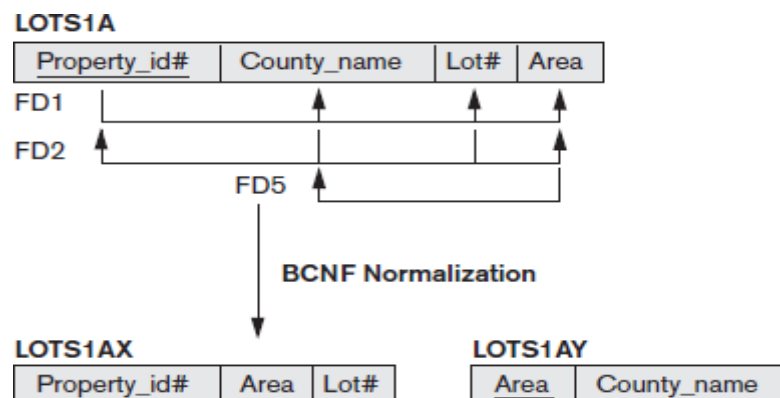


Figure 15.13

Boyce-Codd normal form. (a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition.

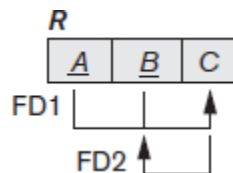
- Consider Figure 15.14, which shows a relation TEACH with the following dependencies:
FD1: {Student, Course} $\rightarrow$ Instructor
FD2: Instructor $\rightarrow$ Course

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

Figure 15.14 A relation TEACH that is in 3NF but not BCNF.

- {Student, Course} is a candidate key for this relation and that the dependencies shown follow the pattern in Figure 15.13(b), with Student as A, Course as B, and Instructor as C. Hence this relation is in 3NF but not BCNF.



- The relation can be decomposed into one of the three following possible pairs:
 - {Student, Instructor} and {Student, Course}.
 - {Course, Instructor} and {Course, Student}.
 - {Instructor, Course} and {Instructor, Student}.
- All three decompositions *lose* the functional dependency FD1. The *desirable* decomposition is (Instructor, Course) and (Instructor, Student), because it is nonadditive join decomposition
- The relation schemas R1 and R2 form a nonadditive join decomposition of R with respect to a set F of functional dependencies if and only if $(R1 \cap R2) \rightarrow (R1 - R2)$ or, $(R1 \cap R2) \rightarrow (R2 - R1)$.

5 . Multivalued Dependency and Fourth Normal Form

- Definition:** A multivalued dependency $X \twoheadrightarrow Y$ specified on relation schema R, where X and Y are both subsets of R, specifies the following constraint on any relation state r of R: If two tuples t1 and t2 exist in r such that $t1[X] = t2[X]$. Then two tuples t3 and t4 should also exist in r with the following properties, where we use Z to denote $(R - (X \cup Y))$
 $t3[X] = t4[X] = t1[X] = t2[X]$.
 $t3[Y] = t1[Y]$ and $t4[Y] = t2[Y]$.
 $t3[Z] = t2[Z]$ and $t4[Z] = t1[Z]$.

- An MVD $X \twoheadrightarrow Y$ in R is called a **trivial MVD** if (a) Y is a subset of X , or (b) $X \cup Y = R$. An MVD that satisfies neither (a) nor (b) is called a **nontrivial MVD**. For example, the relation EMP_PROJECTS in Figure 15.15(b) has the trivial MVD $\text{Ename} \twoheadrightarrow \text{Pname}$.

EMP_PROJECTS

<u>Ename</u>	<u>Pname</u>
Smith	X
Smith	Y

15.15(b)

- Definition.** A relation schema R is in 4NF with respect to a set of dependencies F (that includes functional dependencies and multivalued dependencies) if, for every *nontrivial* multivalued dependency $X \twoheadrightarrow Y$ in F , X is a superkey for R .
- An all-key relation is always in BCNF since it has no FDs. An all-key relation such as the EMP relation in Figure 15.15(a), which has no FDs but has the MVD $\text{Ename} \twoheadrightarrow \text{Pname} \mid \text{Dname}$, is not in 4NF.
- A relation that is not in 4NF due to a nontrivial MVD must be decomposed. To convert it into a set of relations in 4NF, the decomposition removes the redundancy caused by the MVD.
- Consider the EMP relation in Figure 15.15(a). EMP is not in 4NF because of the nontrivial MVDs $\text{Ename} \twoheadrightarrow \text{Pname}$ and $\text{Ename} \twoheadrightarrow \text{Dname}$, and Ename is not a superkey of EMP. We decompose EMP into EMP_PROJECTS and EMP_DEPENDENTS, shown in Figure 15.15(b). Both EMP_PROJECTS and EMP_DEPENDENTS are in 4NF, because the MVDs $\text{Ename} \twoheadrightarrow \text{Pname}$ in EMP_PROJECTS and $\text{Ename} \twoheadrightarrow \text{Dname}$ in EMP_DEPENDENTS are trivial MVDs.

(a) **EMP**

<u>Ename</u>	<u>Pname</u>	<u>Dname</u>
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

(b) **EMP_PROJECTS**

<u>Ename</u>	<u>Pname</u>
Smith	X
Smith	Y

EMP_DEPENDENTS

<u>Ename</u>	<u>Dname</u>
Smith	John
Smith	Anna

Figure 15.15

- (a) The EMP relation with two MVDs: $\text{Ename} \twoheadrightarrow \text{Pname}$ and $\text{Ename} \twoheadrightarrow \text{Dname}$.
 (b) Decomposing the EMP relation into two 4NF relations EMP_PROJECTS and EMP_DEPENDENTS.

- Whenever we decompose a relation schema R into $R_1 = (X \cup Y)$ and $R_2 = (R - Y)$ based on an MVD $X \twoheadrightarrow Y$ that holds in R , the decomposition has the nonadditive join property.
- The following algorithm shows Relational Decomposition into 4NF Relations with Nonadditive Join Property.

Input: A universal relation R and a set of functional and multivalued dependencies F .

1. Set $D := \{ R \}$;
2. While there is a relation schema Q in D that is not in 4NF, do
 - { choose a relation schema Q in D that is not in 4NF;
 - find a nontrivial MVD $X \twoheadrightarrow Y$ in Q that violates 4NF;
 - replace Q in D by two relation schemas $(Q - Y)$ and $(X \cup Y)$;
 - };

6. Join Dependencies and Fifth Normal Form

- A relation schema R when divided into R_1 and R_2 has the lossless property and if the natural join is applied ($R_1 * R_2$) we will get the original relation R .
- In some cases there may be no lossless join decomposition of R if the number of decomposition is equal to two. But if the same relation is decomposed into more than two relations we have a lossless decomposition. This dependency depends on the number of decomposition and hence referred as join dependency
- **Definition.** A **join dependency (JD)**, denoted by $JD(R_1, R_2, \dots, R_n)$, specified on relation schema R , specifies a constraint on the states r of R . The constraint states that every legal state r of R should have a nonadditive join decomposition into $R_1, R_2, \dots, R_n$. Hence, for every such r we have $*(\pi_{R_1}(r), \pi_{R_2}(r), \dots, \pi_{R_n}(r)) = r$.
- A join dependency $JD(R_1, R_2, \dots, R_n)$, specified on relation schema R , is a **trivial JD** if one of the relation schemas R_i in $JD(R_1, R_2, \dots, R_n)$ is equal to R . Such a dependency is called trivial because it has the nonadditive join property for any relation state r of R and thus does not specify any constraint on R .
- **Definition.** A relation schema R is in **fifth normal form (5NF)** (or **project-join normal form (PJNF)**) with respect to a set F of functional, multivalued, and join dependencies if, for every nontrivial join dependency $JD(R_1, R_2, \dots, R_n)$ in F^+ , every R_i is a superkey of R .

SUPPLY

<u>Sname</u>	<u>Part_name</u>	<u>Proj_name</u>
Smith	Bolt	ProjX
Smith	Nut	ProjY
Adamsky	Bolt	ProjY
Walton	Nut	ProjZ
Adamsky	Nail	ProjX
Adamsky	Bolt	ProjX
Smith	Bolt	ProjY

Figure 15.15 (c) The relation SUPPLY with no MVDs is in 4NF but not in 5NF if it has the $JD(R_1, R_2, R_3)$.

R_1		R_2		R_3	
<u>Sname</u>	<u>Part_name</u>	<u>Sname</u>	<u>Proj_name</u>	<u>Part_name</u>	<u>Proj_name</u>
Smith	Bolt	Smith	ProjX	Bolt	ProjX
Smith	Nut	Smith	ProjY	Nut	ProjY
Adamsky	Bolt	Adamsky	ProjY	Bolt	ProjY
Walton	Nut	Walton	ProjZ	Nut	ProjZ
Adamsky	Nail	Adamsky	ProjX	Nail	ProjX

Figure 15.15 (d) Decomposing the relation SUPPLY into the 5NF relations R_1 , R_2 , R_3 .

- For example of a JD, consider once again the SUPPLY all-key relation in Figure 15.15(c). Suppose that the following additional constraint always holds: Whenever a supplier s supplies part p , and a project j uses part p , and the supplier s supplies *at least one* part to project j , then supplier s will also be supplying part p to project j . This constraint can be restated in other ways and specifies a join dependency $JD(R_1, R_2, R_3)$ among the three projections $R_1(\text{Sname}, \text{Part_name})$, $R_2(\text{Sname}, \text{Proj_name})$, and $R_3(\text{Part_name}, \text{Proj_name})$ of SUPPLY.
- Figure 15.15(d) shows how the SUPPLY relation with the join dependency is decomposed into three relations R_1 , R_2 , and R_3 that are each in 5NF. Notice that applying a natural join to any two of these relations produces spurious tuples, but applying a natural join to all three together does not.

SQL:

SQL data definition and data types, specifying constraints in SQL, retrieval queries in SQL, INSERT, DELETE, and UPDATE statements in SQL, Additional features of SQL.

SQL

2.8 SQL Data Definition and Data Types

- ✓ SQL uses the terms table, row, and column for the formal relational model terms relation, tuple, and attribute, respectively.
- ✓ SQL command for data definition is the CREATE statement, which can be used to create schemas, tables (relations), types, and domains, as well as other constructs such as views, assertions, and triggers.
- ✓ An SQL schema is identified by a **schema name** and includes **an authorization identifier** to indicate the user or account who owns the schema, as well as descriptors for each element in the schema.
- ✓ Schema elements include tables, types, constraints, views, domains, and other constructs.
- ✓ A schema is created via the CREATE SCHEMA statement, which can include all the schema elements' definitions.
Eg: CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
- ✓ In SQL **catalog**—a named collection of schemas.

The CREATE TABLE Command in SQL

- ✓ The CREATE TABLE command is used to specify a new relation by giving it a name and specifying its attributes and initial constraints.
- ✓ The attributes are specified first, and each attribute is given a name, a data type to specify its domain of values, and possibly attribute constraints, such as NOT NULL.

- ✓ The key, entity integrity, and referential integrity constraints can be specified within the CREATE TABLE statement after the attributes are declared, or they can be added later using the ALTER TABLE command.
- ✓ Eg: CREATE TABLE EMPLOYEE

```

CREATE TABLE EMPLOYEE
( Fname          VARCHAR(15)          NOT NULL,
  Minit          CHAR,
  Lname          VARCHAR(15)          NOT NULL,
  Ssn            CHAR(9)              NOT NULL,
  Bdate          DATE,
  Address        VARCHAR(30),
  Sex            CHAR,
  Salary         DECIMAL(10,2),
  Super_ssn      CHAR(9),
  Dno            INT                  NOT NULL,
  PRIMARY KEY (Ssn),
CREATE TABLE DEPARTMENT
( Dname          VARCHAR(15)          NOT NULL,
  Dnumber        INT                  NOT NULL,
  Mgr_ssn        CHAR(9)              NOT NULL,
  Mgr_start_date DATE,
  PRIMARY KEY (Dnumber),
  UNIQUE (Dname),
  FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
( Dnumber        INT                  NOT NULL,
  Dlocation       VARCHAR(15)         NOT NULL,
  PRIMARY KEY (Dnumber, Dlocation),
  FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE PROJECT
( Pname          VARCHAR(15)          NOT NULL,
  Pnumber        INT                  NOT NULL,
  Plocation       VARCHAR(15),
  Dnum            INT                  NOT NULL,
  PRIMARY KEY (Pnumber),
  UNIQUE (Pname),
  FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE WORKS_ON
( Essn            CHAR(9)              NOT NULL,
  Pno             INT                  NOT NULL,
  Hours           DECIMAL(3,1)        NOT NULL,
  PRIMARY KEY (Essn, Pno),
  FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn),
  FOREIGN KEY (Pno) REFERENCES PROJECT(Pnumber) );
CREATE TABLE DEPENDENT
( Essn            CHAR(9)              NOT NULL,
  Dependent_name  VARCHAR(15)         NOT NULL,
  Sex             CHAR,
  Bdate           DATE,
  Relationship     VARCHAR(8),
  PRIMARY KEY (Essn, Dependent_name),
  FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn) );

```

Attribute Data Types and Domains in SQL

- ✓ The basic data types available for attributes include numeric, character string, bit string, Boolean, date, and time.

Numeric data types

- Integer numbers of various sizes like INTEGER or INT, and SMALLINT

- Floating-point (real) numbers of various precision like FLOAT or REAL, and DOUBLE PRECISION
- Formatted numbers using DECIMAL(i, j)—or DEC(i, j) or NUMERIC(i, j)—where i, the precision, is the total number of decimal digits and j, the scale, is the number of digits after the decimal point.

Character-string datatypes

- Fixed length—CHAR(n) or CHARACTER(n), where n is the number of character
- Varying length— VARCHAR(n) or CHAR VARYING(n) or CHARACTER VARYING(n), where n is the maximum number of characters.

Bit-string data types

- Fixed length n—BIT(n)—or varying length— BIT VARYING(n), where n is the maximum number of bits.
- Another variable-length bitstring data type called **BINARY LARGE OBJECT** or **BLOB** is also available to specify columns that have large binary values, such as images.

Boolean data types

- Values of TRUE or FALSE.
- In SQL, because of the presence of NULL values, a three-valued logic is used, so a third possible value for a Boolean data type is UNKNOWN.

DATE data types

- Date has ten positions, and its components are YEAR, MONTH, and DAY in the form YYYY-MM-DD.
- The TIME data type has at least eight positions, with the components HOUR, MINUTE, and SECOND in the form HH:MM:SS.

Timestamp data type (TIMESTAMP)

- Includes the DATE and TIME fields, plus a minimum of six positions for decimal fractions of seconds and an optional WITH TIME ZONE qualifier.

Interval

- Another data type related to DATE, TIME, and TIMESTAMP is the INTERVAL data type.
- This specifies an interval—a relative value that can be used to increment or decrement an absolute value of a date, time, or timestamp.

2.9 Specifying Constraints in SQL

These include key and referential integrity constraints, restrictions on attribute domains and NULLs, and constraints on individual tuples within a relation using the CHECK clause.

Specifying Attribute Constraints and Attribute Defaults

- ✓ Because SQL allows NULLs as attribute values, a constraint NOT NULL may be specified if NULL is not permitted for a particular attribute which are part of the primary key of each relation.
- ✓ It is also possible to define a default value for an attribute by appending the clause DEFAULT to an attribute definition.
- ✓ The default value is included in any new tuple if an explicit value is not provided for that attribute.
- ✓ Figure illustrates an example of specifying a default manager for a new department and a default department for a new employee.
- ✓ If no **default clause** is specified, the default *default* value is NULL for attributes that do not have the NOT NULL constraint.
- ✓ Another type of constraint can restrict attribute or domain values using the **CHECK clause** following an attribute or domain definition.
- ✓ For example, suppose that department numbers are restricted to integer numbers between 1 and 20; then, we can change the attribute declaration of Dnumber in the DEPARTMENT table to the following:
Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);

```

CREATE TABLE EMPLOYEE
(
    ...,
    Dno      INT          NOT NULL      DEFAULT 1,
    CONSTRAINT EMPPK
        PRIMARY KEY (Ssn),
    CONSTRAINT EMPSUPERFK
        FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
            ON DELETE SET NULL      ON UPDATE CASCADE,
    CONSTRAINT EMPDEPTFK
        FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
            ON DELETE SET DEFAULT    ON UPDATE CASCADE);

CREATE TABLE DEPARTMENT
(
    ...,
    Mgr_ssn  CHAR(9)      NOT NULL      DEFAULT '888665555',
    ...,
    CONSTRAINT DEPTPK
        PRIMARY KEY (Dnumber),
    CONSTRAINT DEPTSK
        UNIQUE (Dname),
    CONSTRAINT DEPTMGRFK
        FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
            ON DELETE SET DEFAULT    ON UPDATE CASCADE);

CREATE TABLE DEPT_LOCATIONS
(
    ...,
    PRIMARY KEY (Dnumber, Dlocation),
    FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber)
        ON DELETE CASCADE          ON UPDATE CASCADE);

```

Specifying Key and Referential Integrity Constraints

- ✓ The PRIMARY KEY clause specifies one or more attributes that make up the primary key of a relation. If a primary key has a single attribute, the clause can follow the attribute directly.
- ✓ Ex: the primary key of DEPARTMENT can be specified as follows:
Dnumber INT PRIMARY KEY,
- ✓ The UNIQUE clause specifies alternate (unique) keys, also known as candidate keys
- ✓ The UNIQUE clause can also be specified directly for a unique key if it is a single attribute, as in the following example: Dname VARCHAR(15) UNIQUE,
- ✓ Referential integrity is specified via the FOREIGN KEY clause, constraint can be violated when tuples are inserted or deleted, or when a foreign key or primary key attribute value is updated.
- ✓ The default action that SQL takes for an integrity violation is to reject the update operation that will cause a violation, which is known as the RESTRICT option.
- ✓ The schema designer can specify an alternative action to be taken by attaching a referential triggered action clause to any foreign key constraint. The options include SET NULL, CASCADE, and SET DEFAULT. An option must be qualified with either ON DELETE or ON UPDATE.
- ✓ In general, the action taken by the DBMS for SET NULL or SET DEFAULT is the same for both ON DELETE and ON UPDATE: The value of the affected referencing attributes is changed to NULL for SET NULL and to the specified default value of the Key and referential integrity constraints were not included in early versions of SQL.

- ✓ The action for CASCADE ON DELETE is to delete all the referencing tuples, whereas the action for CASCADE ON UPDATE is to change the value of the referencing foreign key attribute(s) to the updated (new) primary key value for all the referencing tuples.

Giving Names to Constraints

- ✓ Figure illustrates how a constraint may be given a constraint name, following the keyword CONSTRAINT.
- ✓ The names of all constraints within a particular schema must be unique.
- ✓ A constraint name is used to identify a particular constraint in case the constraint must be dropped later and replaced with another constraint.

Specifying Constraints on Tuples Using CHECK

- ✓ other table constraints can be specified through additional CHECK clauses at the end of a CREATE TABLE statement.
- ✓ These can be called row-based constraints because they apply to each row individually and are checked whenever a row is inserted or modified.
- ✓ For example, suppose that the DEPARTMENT table in Figure had an additional attribute Dept_create_date, which stores the date when the department was created.
- ✓ In CREATE TABLE statement for the DEPARTMENT table to make sure that a manager's start date is later than the department creation date.
CHECK (Dept_create_date <= Mgr_start_date);

2.10 Basic Retrieval Queries in SQL

- ✓ The basic form of the SELECT statement, sometimes called a mapping or a select-from-where block, is formed of the three clauses SELECT, FROM, and WHERE and has the following form:

SELECT <attribute list>

FROM <table list>

WHERE <condition> ;

where <attribute list> is a list of attribute names whose values are to be retrieved by the query.

<table list> is a list of the relation names required to process the query.

<condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query, the basic logical comparison operators are =, <=, >, >=, and <>

Query 0

Retrieve the birthdate and address of the employee(s) whose name is 'John B Smith'

```
SELECT    BDATE, ADDRESS
FROM      EMPLOYEE
WHERE     FNAME = 'John' AND MINIT = 'B' AND LNAME = 'Smith';
```

Query 1

Retrieve the name and address of all employees who work for the 'Research' department

```
SELECT    FNAME, LNAME, ADDRESS
FROM      EMPLOYEE, DEPARTMENT
WHERE     DNAME = 'Research' AND DNUMBER = DNO;
```

Query 1A

Ambiguous attribute names

```
SELECT    FNAME, EMPLOYEE.NAME, ADDRESS
FROM      EMPLOYEE, DEPARTMENT
WHERE     DEPARTMENT.NAME = 'Research' AND
           DEPARTMENT.DNUMBER = EMPLOYEE.DNUMBER;
```

Query 1B

Aliasing

```
SELECT    E.FNAME, E.NAME, E.ADDRESS
FROM      EMPLOYEE E, DEPARTMENT D
WHERE     D.NAME = 'Research' AND D.DNUMBER = E.DNUMBER;
```

Query 1C

Retrieve all the attribute values of EMPLOYEE tuples who work in department number 5

```
SELECT    *
FROM      EMPLOYEE
WHERE     DNO = 5;
```

Query 1D

Retrieve all the attributes of an EMPLOYEE and the attributes of the DEPARTMENT he or she works in for every employee of the 'Research' department

```
SELECT    *
FROM      EMPLOYEE, DEPARTMENT
WHERE     DNAME = 'Research' AND DNO = DNUMBER;
```

Query 2

For every project located in 'Stafford', list the project number, the controlling department number and the department manager's last name, address and birthdate

```
SELECT    PNUMBER, DNUM, LNAME, ADDRESS, BDATE
FROM      PROJECT, DEPARTMENT, EMPLOYEE
WHERE     DNUM = DNUMBER AND MGRSSN = SSN AND PLOCATION = 'Stafford';
```

Query 8

For each employee, retrieve the employee's first and last name, and the first and last name of his or her immediate supervisor.

```
SELECT    E.FNAME, E.LNAME, S.FNAME, S.LNAME
FROM      EMPLOYEE AS E, EMPLOYEE AS S
WHERE     E.SUPERSSN = S.SSN;
```

Query 8A

Reformulation of query 8 to retrieve the last name of each employee and his or her supervisor, while renaming the resulting attribute names as EMPLOYEE_NAME and SUPERVISOR_NAME

```
SELECT    E.LNAME AS EMPLOYEE_NAME, S.LNAME AS SUPERVISOR_NAME
FROM      EMPLOYEE AS E, EMPLOYEE AS S
WHERE     E.SUPERSSN = S.SSN;
```

Query 9

Select all EMPLOYEE SSNs in the database

```
SELECT    SSN
FROM      EMPLOYEE;
```

Query 10

Select all combination of EMPLOYEE SSN and DEPARTMENT DNAME in the database

```
SELECT    SSN, DNAME
FROM      EMPLOYEE, DEPARTMENT;
```

Query 10A

Select the CROSS PRODUCT of the EMPLOYEE and DEPARTMENT relations

```
SELECT    *
FROM      EMPLOYEE, DEPARTMENT;
```

Query 11

Retrieve the salary of every employee

```
SELECT ALL    SALARY
FROM          EMPLOYEE;
```

Query 11A

Retrieve all distinct salary values

```
SELECT DISTINCT    SALARY
FROM                EMPLOYEE;
```

Query 12

Retrieve all employees whose address is in Houston, Texas

```
SELECT    FNAME, LNAME
FROM      EMPLOYEE
WHERE     ADDRESS LIKE '%Houston,TX%';
```

Query 12A

Find all employees who were born during the 1950s

```
SELECT    FNAME, LNAME
FROM      EMPLOYEE
WHERE     BDATE LIKE '_____ 70';
```

Query 13

Show the resulting salaries if every employee working on the 'ProductX' project is given a 10 percent raise

```
SELECT    FNAME, LNAME, 1.1*SALARY
FROM      EMPLOYEE, WORKS_ON, PROJECT
WHERE     SSN = ESSN AND PNO = PNUMBER AND PNAME = 'ProductX';
```

Query 14

Retrieve all employees in department 5 whose salary is between £30,000 and £40,000

```

SELECT      *
FROM        EMPLOYEE
WHERE       (SALARY BETWEEN 30000 AND 40000) AND DNO = 5;

```

Query 15

Retrieve a list of employees and the projects they are working on, ordered by department and, within each department, ordered alphabetically by last name, first name

```

SELECT      DNAME, LNAME, FNAME, PNAME
FROM        DEPARTMENT, EMPLOYEE, WORKS_ON, PROJECT
WHERE       DNUMBER = DNO AND SSN = ESSN AND PNO = PNUMBER
ORDER BY    DNAME DESC, LNAME ASC, FNAME ASC;

```

2.11 INSERT, DELETE, and UPDATE Statements in SQL

The INSERT Command

- ✓ INSERT is used to add a single tuple (row) to a relation (table).
- ✓ must specify the relation name and a list of values for the tuple.
- ✓ The values should be listed in the same order in which the corresponding attributes were specified in the CREATE TABLE command.
- ✓ INSERT INTO EMPLOYEE VALUES ('Richard', 'K', 'Marini', '653298653', '1962-12-30', '98 Oak Forest, Katy, TX', 'M', 37000, '653298653', 4);
- ✓ For example, to enter a tuple for a new EMPLOYEE for whom we know only the Fname, Lname, Dno, and Ssn attributes,
 INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn) VALUES ('Richard', 'Marini', 4, '653298653');

The DELETE Command

- ✓ The DELETE command removes tuples from a relation.
- ✓ It includes a WHERE clause, similar to that used in an SQL query, to select the tuples to be deleted.
- ✓ Tuples are explicitly deleted from only one table at a time.
- ✓ Depending on the number of tuples selected by the condition in the WHERE clause, zero, one, or several tuples can be deleted by a single DELETE command.
- ✓ A missing WHERE clause specifies that all tuples in the relation are to be deleted.
- ✓ DELETE FROM EMPLOYEE WHERE Ssn = '123456789';
- ✓ DELETE FROM EMPLOYEE;

The UPDATE Command

- ✓ The UPDATE command is used to modify attribute values of one or more selected tuples.
- ✓ As in the DELETE command, a WHERE clause in the UPDATE command selects the tuples to be modified from a single relation.
- ✓ UPDATE PROJECT,
 SET Plocation = 'Bellaire', Dnum = 5
 WHERE Pnumber = 10;

2.12 Additional Features of SQL

- ✓ various techniques for specifying complex retrieval queries, including nested queries, aggregate functions, grouping, joined tables, outer joins, case statements, and recursive queries; SQL **views, triggers, and assertions**; and commands for schema modification.
- ✓ SQL has various techniques for writing programs in various programming languages that include SQL statements to access one or more databases. These include **embedded** (and dynamic) SQL, **SQL/CLI** (Call Level Interface) and its predecessor **ODBC** (Open Data Base Connectivity), and **SQL/PSM** (Persistent Stored Modules).
- ✓ Each commercial RDBMS will have, in addition to the SQL commands, a set of commands for specifying physical database design parameters, file structures for relations, and access paths such as indexes. We called these commands a storage definition language (SDL).
- ✓ SQL has **transaction control commands**. These are used to specify units of database processing for concurrency control and recovery purposes.
- ✓ SQL has language constructs for specifying the granting and revoking of privileges to users. Privileges typically correspond to the right to use certain SQL commands to access certain relations. Each relation is assigned an owner, and either the owner or the DBA staff can grant to selected users the privilege to use an SQL statement—such as SELECT INSERT, DELETE, or UPDATE—to access the relation. In addition, the DBA staff can grant the privileges to create schemas, tables, or views to certain users. These SQL commands—called **GRANT and REVOKE**.
- ✓ SQL has language constructs for **creating triggers**. These are generally referred to as active database techniques, since they specify actions that are automatically triggered by events such as database updates.
- ✓ SQL has incorporated many features from **object-oriented models** to have more powerful capabilities, leading to enhanced relational systems known as object-relational. Capabilities such as creating complex-structured attributes, specifying abstract data types (called UDTs or user-defined types) for attributes and tables, creating object identifiers for referencing tuples, and specifying operations on types.
- ✓ SQL and relational databases can interact with new technologies such as **XML** and OLAP/data warehouses.